JAVASCRIPT BASICS



JAVASCRIPT

- linguaggio compilato just-in-time
- eseguito nel browser (ma anche alcuni server hanno un motore js)
- può accedere e modificare il document object model (dom)



COME AGGIUNGERE JS

JavaScript può essere aggiunto a un documento HTML in tre modi:

1. **Interno/Inline**: All'interno del tag `<script>` nella sezione `<head>` o `<body>`.
2. **Esterno**: Collegando un file `.js` esterno usando il tag `<script src="file.js"></script>`.
    - **Best Practice**: Collocare gli script esterni prima della chiusura del tag `</body>` per migliorare la velocità di caricamento.
3. **HTML Event Handler**: Direttamente negli attributi HTML (es. `onclick="..."`), sebbene questa pratica sia sconsigliata.



STATEMENT E COMMENTI

- **Statement**: Le istruzioni JavaScript sono comandi per essere eseguiti dal browser.
- **Separatore**: Normalmente ogni statement termina con un punto e virgola `;`. La sua omissione è permessa ma sconsigliata (Automatic Semicolon Insertion).
- **Case Sensitivity**: JavaScript distingue tra maiuscole e minuscole.



```
let a = 1; // Un statement
let b = 2; // Un altro statement

// Questo è un commento su riga singola
/* Questo è un
commento su
più righe
*/
```



TIPIZZAZIONE

- stringhe, numeri, booleani, ...
- oggetti (inclusi gli array)
- tipizzazione dinamica (il tipo di una variabile può cambiare)
- tipizzazione debole (casting a seconda dell'operazione)



VARIABILI

- `var`: con scope di funzione (funzione-scoped)
- `let`: con scope di blocco (blocco-scoped)
- `const`: con scope di blocco, non può essere riassegnata
- assegnazione senza dichiarazione (`x = "foo"`): assegnazione deprecata (scope globale)



```
var a;
var b = 1;
b += 5; // riassegnata, ora è 6

let c;
let d = 3 / 2; // è 1.5
d = "Hello"; // riassegnata, cambiato tipo
d += " world"; // ora "Hello world"
d = d + 1; // ora "Hello world1"

const k = 30; // blocco-scoped, non può essere riassegnata
x = "foo"; // assegnazione deprecata (scope globale)
```



SCOPE DI BLOCCO VS SCOPE DI FUNZIONE

- **Scope di funzione (`var`)**: Le variabili dichiarate con `var` sono accessibili ovunque all'interno della funzione.
- **Scope di blocco (`let`, `const`)**: Le variabili dichiarate con `let` o `const` sono limitate al blocco di codice immediato (delimitato da parentesi graffe `{}`) in cui sono definite.



```
// --- Scope di Funzione (VAR) ---
function scopeFunzione() {
  if (true) {
    var messaggioVar = "Sono all'interno dell'if (var)";
  }
  // Accessibile qui, fuori dal blocco {if}
  console.log(messaggioVar); // logs: "Sono all'interno dell'if (var)"
}
scopeFunzione();

// --- Scope di Blocco (LET) ---
function scopeBlocco() {
  if (true) {
    let messaggioLet = "Sono all'interno dell'if (let)";
    console.log(messaggioLet); // logs: "Sono all'interno dell'if (let)"
  }
  // NON accessibile qui, fuori dal blocco {if}
  // console.log(messaggioLet); // Errore: ReferenceError
}
scopeBlocco();
```

HOISTING (SOLLEVAMENTO)

- JavaScript sposta le dichiarazioni (ma non le inizializzazioni) in cima allo scope in fase di compilazione.
- **`var`**: Sia la dichiarazione che l'inizializzazione implicita a `undefined` vengono sollevate.
- **`let/const`**: La dichiarazione viene sollevata, ma non l'inizializzazione. L'accesso prima della dichiarazione esplicita causa un `ReferenceError` (Temporal Dead Zone).

Quindi se ho definito una const a riga 40 con valore 51, questa const esiste già da riga 0, ma non ha il valore 51

```
// Esempio con VAR
console.log(messaggioA); // logs: undefined (Hoisting di var)
var messaggioA = "Ciao";

// Esempio con LET
// console.log(messaggioB); // Errore: ReferenceError (Temporal Dead Zone)
let messaggioB = "Hello";
```



OPERATORI

- **Aritmetici**: `+`, `-`, `*`, `/`, `%` (modulo), `++` (incremento), `--` (decremento).
- **Assegnazione**: `=`, `+=`, `-=`, `*=`, `/=`, `%=`.
- **Stringa**: `+` (concatenazione).
- **Logici**: `&&` (AND), `||` (OR), `!` (NOT).
- **Comparazione**:
  - `==` (uguaglianza debole, con coercizione di tipo)
  - `===` (uguaglianza stretta, *senza* coercizione di tipo)
  - `!=` (disuguaglianza debole), `!==` (disuguaglianza stretta)
  - `>`, `<`, `>=`, `<=`.

```
let x = 5 + 3; // 8
x += 2; // x è 10

let str = "Hello" + " " + "World"; // "Hello World"

// Esempi di Comparazione
// Uguaglianza Debole (coercizione di tipo)
console.log(5 == "5");   // true
// Uguaglianza Stretta (NO coercizione)
console.log(5 === "5");  // false
```



ISTRUZIONI CONDIZIONALI

Utilizzate per eseguire azioni diverse in base a condizioni diverse.

- `if`: Esegue un blocco di codice se una condizione è vera.
- `else`: Esegue un blocco di codice se la condizione precedente è falsa.
- `else if`: Specifica una nuova condizione da testare se la prima condizione è falsa.
- `switch`: Seleziona uno dei tanti blocchi di codice da eseguire.



```
let tempo = "sole";
if (tempo === "pioggia") {
  console.log("Porta l'ombrello");
} else if (tempo === "neve") {
  console.log("Fai un pupazzo");
} else {
  console.log("Fa bel tempo!"); // Eseguito
}

let giorno = 3;
switch (giorno) {
  case 1:
    console.log("Lunedì");
    break;
  case 7:
    console.log("Domenica");
    break;
  default:
    console.log("Giorno feriale"); // Eseguito
}
```

ISTRUZIONI LOOP

Utilizzate per eseguire ripetutamente lo stesso blocco di codice finché non viene soddisfatta una condizione specifica.

- `for`: Esegue un blocco di codice per un numero predefinito di volte.
- `for/in`: Itera attraverso le proprietà di un oggetto.
- `for/of`: Itera sui valori di un oggetto iterabile (es. Array, String, Map, Set).
- `while`: Esegue un blocco di codice finché una condizione specificata è vera.
- `do/while`: Esegue il blocco di codice una volta, quindi ripete il loop finché la condizione è vera.

```
// For loop standard
for (let i = 0; i < 3; i++) {
  console.log("Numero: " + i);
}

// For/in (per le proprietà dell'oggetto)
const persona = {nome: "Mario", eta: 40};
for (let key in persona) {
  console.log(key + ": " + persona[key]);
}

// While loop
let count = 0;
while (count < 2) {
  console.log("Count: " + count);
  count++;
}
```

OGGETTI

- collezione di proprietà
- mutabile
- normalmente:
  - le chiavi delle proprietà sono stringhe
  - i valori delle proprietà sono di qualsiasi tipo js
- `array`, `function`, `date`, `regexp`, `error`: sono oggetti



FUNZIONI

- possono restituire valori
- `return` opzionale
- se `return` non è specificato, la funzione restituisce `undefined`

```
function myFunction(a, b) {
  return a * b;
}

const x = myFunction(4, 3); // x è 12
```

UNDEFINED

- `undefined` è un tipo di dato primitivo e un valore
- indica che una variabile è stata dichiarata ma non le è stato assegnato un valore
- è anche il valore restituito da una funzione che non usa esplicitamente l'istruzione `return` (o usa `return` senza valore)

```
let a; // a è undefined
console.log(a); // logs: undefined

function logMessage(msg) {
    console.log(msg); // questa funzione non ha return
}

const result = logMessage("ciao");
console.log(result); // logs: undefined (il valore di ritorno implicito)
```

ARROW FUNCTION (FUNZIONI FRECCIA)

```
// sintassi concisa per le funzioni
const sum = (a, b) => {
  return a + b;
};

// sintassi concisa per singola espressione
const square = (a) => a * a;
```



DOCUMENT OBJECT MODEL (DOM)

- modello di programmazione per documenti html e xml
- fornisce una rappresentazione strutturata (ad albero) del documento
- definisce un'interfaccia per interagire con l'albero e manipolare gli elementi



ACCEDERE AGLI ELEMENTI DEL DOM

- `document.getElementById("id")`: trova un elemento per id
- `document.getElementsByTagName("name")`: trova elementi per nome tag (restituisce una `htmlcollection`)
- `document.getElementsByClassName("name")`: trova elementi per nome classe (restituisce una `htmlcollection`)
- `document.querySelector("css selector")`: trova il *primo* elemento che corrisponde a un selettore css
- `document.querySelectorAll("css selector")`: trova *tutti* gli elementi che corrispondono a un selettore css (restituisce una `nodelist`)

MODIFICARE GLI ELEMENTI DEL DOM

- `element.innerHTML = new html content`: modifica il contenuto html di un elemento
- `element.attribute = new value`: modifica il valore di un attributo di un elemento
- `element.style.property = new style`: modifica lo stile di un elemento

```
document.getElementById("my-id").innerHTML = "nuovo testo";
document.getElementById("my-link").href = "new-url.html";
```

AGGIUNGERE E RIMUOVERE ELEMENTI DOM

- `document.createElement(element)`: crea un nuovo elemento html
- `document.removeChild(element)`: rimuove un elemento html
- `document.appendChild(element)`: aggiunge un elemento html
- `document.replaceChild(new, old)`: sostituisce un elemento html
- `document.write(text)`: scrive direttamente nell'output html (raramente usato)



ESEMPIO DI MANIPOLAZIONE DOM

```
// crea un nuovo elemento <span>
const myElem = document.createElement('span');

// imposta l'id, lo stile e il contenuto
myElem.id = 'bar';
myElem.style = "color:red;";
myElem.innerHTML = "questo è rosso";

// aggiunge il nuovo elemento al corpo del documento
document.body.appendChild(myElem);
```



EXPORT

- dichiarazione usata per esportare valori da un modulo javascript
- aggiungendo `type="module"` al tag `<script>`, il runtime interpreta il file html come un modulo
- i valori esportati possono essere importati in altri moduli
- un valore in un binding importato cambierà se cambia nel modulo di esportazione

```
export let name1, name2 /*, ... */; // anche var e const
export function functionName() { /* ... */ }
export { variable1 as name1, variable2 as name2, /* ..., */ nameN };
export default function functionName() { /* ... */ }
export * from "module-name"; // aggregazione di moduli
```

IMPORT

Notare che se le funzioni o variabili importate cambiano nel file originale, cambieranno anche qui

```
// file test.js
const k = 12;
export default k;

// un altro file
import m from './test'; // possiamo usare m invece di k,
                        // perché k era l'export di default
console.log(m); // stamperà 12
```



DEFAULT

La parola chiave **`default`** ha un significato speciale nel sistema di moduli di JavaScript:

1. **Definisce l'Esportazione Principale:** Ogni modulo può avere **una e una sola** esportazione di default. Questa è l'esportazione "più importante" o l'entità principale che ci si aspetta venga utilizzata da chiunque importi il modulo.
2. **Semplifica l'Importazione:** Quando si importa l'esportazione di default, non è necessario usare le parentesi graffe `{}` e si può **scegliere un nome a piacere** per l'entità importata.



Tipo di Export	Sintassi Export	Sintassi Import	Regola Naming
Nominata	`export const foo = 1;`	`import { foo } from './file';`	**Devi** usare lo stesso nome (`foo`).
Default	`const bar = 2; export default bar;`	`import qualsiasiNome from './file';`	**Puoi** usare qualsiasi nome.



REFERENCES

- https://en.wikipedia.org/wiki/JavaScript
- https://www.w3schools.com/js/default.asp
- https://developer.mozilla.org/en...

